

Unified Species Code Namespace

VSEPR-SIM – Code Draft

`include/physics/particle_id.hpp` `legacy/c_api/particle_ids.h`

`include/physics/decay_event_builder.hpp`

v4.1.0 June 2025

Contents

1	Core Idea	2
1.1	Geometry Table Integration	2
2	The Reserved Negative Ladder	2
2.1	Notes on Selected Entries	3
3	particle_id.hpp – Full Listing	3
3.1	Sub-range Predicates	4
3.2	Label and <code>species_code</code> Helpers	5
4	Updated Decay Builders	6
5	Plain C Legacy API	7
6	Query Classification Table	8
7	Geometry Table Dispatch Pattern	8
7.1	Writing a Row	8
7.2	Reading and Dispatching	9
8	Relation to Existing Infrastructure	9

Core Idea

Every row in the engine geometry table carries a single signed integer called `species_code`. The sign tells you immediately what kind of entity the coordinate belongs to:

$$\text{species_code} \in \mathbb{Z}, \quad \begin{cases} \geq 1 & \text{standard atom template } (Z = 1 \dots 118) \\ = 0 & \text{placeholder / unresolved} \\ \leq -1 & \text{reserved engine entity (negative class space)} \end{cases}$$

The negative range is a *reserved class space*: every nonstandard, derived, emitted, virtual, or bookkeeping entity that can appear in the coordinate table gets a permanent negative code. No separate type tag, no string parsing, no secondary lookup.

Geometry Table Integration

A geometry table row has the shape:

sample_id	bead_id	local_id	species_code	label	x	y	z
17	0	0	6	C	0.000	1.402	0.000
17	0	1	1	H	1.050	2.100	0.000
17	1	0	-1	alpha	2.500	0.300	0.200
17	1	1	-3	gamma	2.700	0.300	0.200
17	2	0	-4	eta	3.100	0.100	0.050
17	3	0	-17	ghost	0.000	0.000	0.000

Listing 1: Long-form geometry table with `species_code` column

`species_code` is the master numeric identifier. Positive or zero means standard atom/object template; negative means reserved synthetic class. The `label` column is derived from `species_code` via `species_code_to_label()` and is never the source of truth.

The Reserved Negative Ladder

The negative range is partitioned into three consecutive bands:

$$\begin{array}{ccc} \underbrace{[-1, -8]} & \underbrace{[-9, -14]} & \underbrace{[-15, -18]} \\ \text{physical emission /} & \text{energy / field /} & \text{defect / structural /} \\ \text{decay particles} & \text{abstract carriers} & \text{bookkeeping} \end{array}$$

Code	Enum name	Label	Description
<i>Physical emission / decay particles</i>			
–1	alpha	alpha	He-4 nucleus (alpha decay)
–2	beta_minus	beta-	Electron (beta-minus decay)
–3	gamma	gamma	Gamma photon
–4	eta	eta	QMD energy packet – abstract high-definition energy carrier; extra to the standard kernel energy unit
–5	antineutrino	antineutrino	Antineutrino loss token
–6	neutrino	neutrino	Neutrino loss token
–7	neutron_free	neutron	Free neutron
–8	proton_free	proton	Free proton
<i>Energy / field / abstract carriers</i>			
–9	excitation_packet	excitation	Electronic excitation carrier
–10	ionization_event	ionization	Ionization bookkeeping token
–11	heat_packet	heat	Phonon / thermal energy packet
–12	charge_cloud	charge_cloud	Distributed charge carrier
–13	field_source	field_source	Field-origin sentinel
–14	field_probe	field_probe	Field-sampling sentinel
<i>Defect / structural / bookkeeping</i>			
–15	vacancy	vacancy	Lattice vacancy defect
–16	interstitial	interstitial	Lattice interstitial defect
–17	ghost	ghost	Ghost particle – virtual massless sentinel for field evaluation, constraint anchoring, or higher-order virtual entities
–18	transition_state	transition_state	Reaction coordinate marker

Notes on Selected Entries

–4: **eta packet.** The **eta** entry is an abstract energy packet that lives above the standard kernel energy unit. It is specifically designed for high-definition QMD simulations where sub-particle energy resolution is required. It can appear as an emitted row in the geometry table, carry coordinates, and be tracked like any other entity.

–17: **ghost particle.** The ghost particle is massless and exerts no direct force. Its coordinate is meaningful: it marks a point in space used for field evaluation, constraint anchoring, reaction coordinate sampling, or as a stand-in for higher-order virtual entities. A ghost row in the geometry table is valid geometry; it never contributes to pair interactions.

particle_id.hpp – Full Listing

```

1 namespace vsepr::physics {
2
3 enum class ParticleID : std::int32_t {
4     placeholder          = 0,
5
6     // Physical emission / decay particles

```

```

7     alpha                = -1,
8     beta_minus           = -2,
9     gamma                = -3,
10    eta                  = -4,    // QMD energy packet
11    antineutrino          = -5,
12    neutrino              = -6,
13    neutron_free          = -7,
14    proton_free           = -8,
15
16    // Energy / field / abstract carriers
17    excitation_packet     = -9,
18    ionization_event      = -10,
19    heat_packet           = -11,
20    charge_cloud          = -12,
21    field_source          = -13,
22    field_probe           = -14,
23
24    // Defect / structural / bookkeeping
25    vacancy               = -15,
26    interstitial          = -16,
27    ghost                 = -17,
28    transition_state      = -18
29 };
30
31 } // namespace vsepr::physics

```

Listing 2: include/physics/particle_id.hpp – enum class

Sub-range Predicates

```

1 // Full engine-reserved range [0, -99]
2 [[nodiscard]] constexpr bool is_reserved(ParticleID id) noexcept {
3     const auto v = static_cast<std::int32_t>(id);
4     return (v <= 0 && v >= -99);
5 }
6
7 // Physical decay / emission particles [-1, -8]
8 [[nodiscard]] constexpr bool is_decay_particle(ParticleID id) noexcept {
9     const auto v = static_cast<std::int32_t>(id);
10    return (v >= -8 && v <= -1);
11 }
12
13 // Physically propagates through matter
14 [[nodiscard]] constexpr bool is_transportable(ParticleID id) noexcept {
15     switch (id) {
16     case ParticleID::alpha:
17     case ParticleID::beta_minus:
18     case ParticleID::gamma:
19     case ParticleID::neutron_free:
20     case ParticleID::proton_free:
21         return true;
22     default: return false;
23     }
24 }
25
26 // Never transported -- accounting token only
27 [[nodiscard]] constexpr bool is_bookkeeping_only(ParticleID id) noexcept {
28     switch (id) {
29     case ParticleID::eta:
30     case ParticleID::antineutrino:
31     case ParticleID::neutrino:
32     case ParticleID::excitation_packet:
33     case ParticleID::ionization_event:

```

```

34     case ParticleID::heat_packet:
35     case ParticleID::charge_cloud:
36     case ParticleID::field_source:
37     case ParticleID::field_probe:
38     case ParticleID::transition_state:
39         return true;
40     default: return false;
41 }
42 }
43
44 // Energy / field abstract carriers [-9, -14]
45 [[nodiscard]] constexpr bool is_energy_carrier(ParticleID id) noexcept {
46     const auto v = static_cast<std::int32_t>(id);
47     return (v >= -14 && v <= -9);
48 }
49
50 // Defect / structural / virtual entities [-15, -18]
51 [[nodiscard]] constexpr bool is_defect_or_virtual(ParticleID id) noexcept {
52     const auto v = static_cast<std::int32_t>(id);
53     return (v >= -18 && v <= -15);
54 }
55
56 // Ghost particle specifically
57 [[nodiscard]] constexpr bool is_ghost(ParticleID id) noexcept {
58     return id == ParticleID::ghost;
59 }

```

Listing 3: Constexpr sub-range predicates

Label and species_code Helpers

```

1  [[nodiscard]] constexpr std::string_view to_string(ParticleID id) noexcept {
2      switch (id) {
3          case ParticleID::placeholder:      return "X";
4          case ParticleID::alpha:            return "alpha";
5          case ParticleID::beta_minus:      return "beta-";
6          case ParticleID::gamma:           return "gamma";
7          case ParticleID::eta:             return "eta";
8          case ParticleID::antineutrino:    return "antineutrino";
9          case ParticleID::neutrino:        return "neutrino";
10         case ParticleID::neutron_free:    return "neutron";
11         case ParticleID::proton_free:     return "proton";
12         case ParticleID::excitation_packet: return "excitation";
13         case ParticleID::ionization_event: return "ionization";
14         case ParticleID::heat_packet:     return "heat";
15         case ParticleID::charge_cloud:    return "charge_cloud";
16         case ParticleID::field_source:    return "field_source";
17         case ParticleID::field_probe:     return "field_probe";
18         case ParticleID::vacancy:         return "vacancy";
19         case ParticleID::interstitial:    return "interstitial";
20         case ParticleID::ghost:           return "ghost";
21         case ParticleID::transition_state: return "transition_state";
22         default:                          return "unknown";
23     }
24 }
25
26 // Raw int32_t interface for geometry table I/O.
27 // Positive codes (atom Z) return "?" -- caller resolves via periodic table.
28 [[nodiscard]] constexpr std::string_view
29 species_code_to_label(std::int32_t code) noexcept {
30     if (code > 0) return "?";
31     return to_string(static_cast<ParticleID>(code));
32 }

```

```

33
34 // True if code is a standard atom template (Z in [1, 118])
35 [[nodiscard]] constexpr bool is_atom_species(std::int32_t code) noexcept {
36     return (code >= 1 && code <= 118);
37 }
38
39 // True if code is a reserved engine entity (code <= 0)
40 [[nodiscard]] constexpr bool is_reserved_code(std::int32_t code) noexcept {
41     return (code <= 0 && code >= -99);
42 }

```

Listing 4: Geometry table label generation

Updated Decay Builders

The builder factories emit particles using the new enum names. Beta-minus decay appends antineutrino (code -5) specifically rather than a generic loss token.

```

1 // S(Z,A) -> S(Z-2, A-4) + alpha
2 [[nodiscard]] inline DecayEvent make_alpha_decay(
3     std::int32_t parent, std::int32_t daughter,
4     double energy_eV, double time_s, double confidence
5 ) {
6     DecayEvent ev {};
7     ev.parent_species_id = parent;
8     ev.daughter_species_id = daughter;
9     ev.released_energy_eV = energy_eV;
10    ev.event_time_s = time_s;
11    ev.confidence = confidence;
12    ev.deposited_energy_fraction = 0.85;
13    ev.transported_energy_fraction = 0.15;
14    ev.local_damage_score = 0.90;
15    (void)ev.add_emitted(ParticleID::alpha); // -1
16    return ev;
17 }
18
19 // S(Z,A) -> S(Z+1, A) + beta_minus + antineutrino
20 [[nodiscard]] inline DecayEvent make_beta_minus_decay(
21     std::int32_t parent, std::int32_t daughter,
22     double energy_eV, double time_s, double confidence
23 ) {
24     DecayEvent ev {};
25     ev.parent_species_id = parent;
26     ev.daughter_species_id = daughter;
27     ev.released_energy_eV = energy_eV;
28     ev.event_time_s = time_s;
29     ev.confidence = confidence;
30     ev.deposited_energy_fraction = 0.35;
31     ev.transported_energy_fraction = 0.65;
32     ev.local_damage_score = 0.30;
33     (void)ev.add_emitted(ParticleID::beta_minus); // -2
34     (void)ev.add_emitted(ParticleID::antineutrino); // -5
35     return ev;
36 }
37
38 // S*(Z,A) -> S(Z,A) + gamma
39 [[nodiscard]] inline DecayEvent make_gamma_decay(
40     std::int32_t parent, std::int32_t daughter,
41     double energy_eV, double time_s, double confidence
42 ) {
43     DecayEvent ev {};
44     ev.parent_species_id = parent;

```

```

45     ev.daughter_species_id = daughter;
46     ev.released_energy_eV  = energy_eV;
47     ev.event_time_s       = time_s;
48     ev.confidence         = confidence;
49     ev.deposited_energy_fraction = 0.10;
50     ev.transported_energy_fraction = 0.90;
51     ev.local_damage_score  = 0.05;
52     (void)ev.add_emitted(ParticleID::gamma);           // -3
53     return ev;
54 }

```

Listing 5: Alpha, beta-minus, and gamma decay factories

Plain C Legacy API

```

1  typedef enum ParticleID {
2      PARTICLE_PLACEHOLDER          = 0,
3
4      /* Physical emission / decay particles */
5      PARTICLE_ALPHA                 = -1,
6      PARTICLE_BETA_MINUS            = -2,
7      PARTICLE_GAMMA                 = -3,
8      PARTICLE_ETA                   = -4,      /* QMD energy packet */
9      PARTICLE_antineutrino          = -5,
10     PARTICLE_NEUTRINO               = -6,
11     PARTICLE_NEUTRON_FREE           = -7,
12     PARTICLE_PROTON_FREE            = -8,
13
14     /* Energy / field / abstract carriers */
15     PARTICLE_EXCITATION              = -9,
16     PARTICLE_IONIZATION              = -10,
17     PARTICLE_HEAT_PACKET             = -11,
18     PARTICLE_CHARGE_CLOUD            = -12,
19     PARTICLE_FIELD_SOURCE            = -13,
20     PARTICLE_FIELD_PROBE             = -14,
21
22     /* Defect / structural / virtual */
23     PARTICLE_VACANCY                 = -15,
24     PARTICLE_INTERSTITIAL            = -16,
25     PARTICLE_GHOST                   = -17,
26     PARTICLE_TRANSITION_STATE        = -18
27 } ParticleID;
28
29 const char* particle_id_name(int id);
30 int particle_id_is_reserved(int id);
31 int particle_id_is_transportable(int id);
32 int particle_id_is_bookkeeping_only(int id);
33 int particle_id_is_atom_species(int id);

```

Listing 6: legacy/c_api/particle_ids.h – updated C enum

Query Classification Table

Code	Name	<i>reserved</i>	<i>decay_particle</i>	<i>transportable</i>	<i>energy_carrier</i>	<i>defect_or_virtual</i>
-1	alpha	•	•	•		
-2	beta_minus	•	•	•		
-3	gamma	•	•	•		
-4	eta	•	•			
-5	antineutrino	•	•			
-6	neutrino	•	•			
-7	neutron_free	•	•	•		
-8	proton_free	•	•	•		
-9	excitation_packet	•			•	
-10	ionization_event	•			•	
-11	heat_packet	•			•	
-12	charge_cloud	•			•	
-13	field_source	•			•	
-14	field_probe	•			•	
-15	vacancy	•				•
-16	interstitial	•				•
-17	ghost	•				•
-18	transition_state	•				•

Note: `is_bookkeeping_only` returns true for all entries where `is_transportable` is false and `is_reserved` is true, i.e. codes -4 through -6, codes -9 through -14, and -18.

Geometry Table Dispatch Pattern

Writing a Row

```

1 void write_row(std::ostream& out,
2               int sample_id, int bead_id, int local_id,
3               std::int32_t species_code,
4               double x, double y, double z,
5               std::string_view state)
6 {
7     using namespace vsepr::physics;
8
9     std::string_view label;
10    if (is_atom_species(species_code)) {
11        label = element_symbol(species_code); // periodic table
12    } else {
13        label = species_code_to_label(species_code);
14    }
15
16    out << sample_id << '\t'
17        << bead_id << '\t'
18        << local_id << '\t'

```



```

19         << species_code << '\t'
20         << label         << '\t'
21         << x << '\t' << y << '\t' << z << '\t'
22         << state         << '\n';
23     }
24
25     // Examples:
26     write_row(out, 17, 0, 0, 6, x, y, z, "final"); // Carbon
27     write_row(out, 17, 1, 0, -1, x, y, z, "emitted"); // alpha
28     write_row(out, 17, 2, 0, -4, x, y, z, "emitted"); // eta
29     write_row(out, 17, 3, 0, -17, x, y, z, "virtual"); // ghost

```

Listing 7: Writing a geometry table row for any species code

Reading and Dispatching

```

1 void dispatch_row(std::int32_t sc, double x, double y, double z)
2 {
3     using namespace vsepr::physics;
4     const auto id = static_cast<ParticleID>(sc);
5
6     if (is_atom_species(sc)) {
7         add_atom(sc, x, y, z); // force-field
8     }
9     else if (is_decay_particle(id)) {
10        register_emitted(sc, x, y, z); // transport
11    }
12    else if (is_energy_carrier(id)) {
13        register_energy_token(sc, x, y, z); // accounting
14    }
15    else if (is_ghost(id)) {
16        register_field_point(x, y, z); // no force
17    }
18    else if (is_defect_or_virtual(id)) {
19        register_defect(sc, x, y, z); // structural
20    }
21 }

```

Listing 8: Dispatch on species_code while reading geometry table

Relation to Existing Infrastructure

Component	Location	Role
vsepr::nuclear::DecayMode	decay_chains.hpp	How a chain step progresses
atomistic::nuclear::DecayType	nuclear_stability.hpp	Dominant mode prediction for Z
vsepr::physics::ParticleID	particle_id.hpp	Integer code in the geometry row
XYZAtom::atom_type	xyz_format.hpp	Force-field type (always positive)

DecayMode and DecayType answer “what kind of transition occurred?” ParticleID / species_code answers “what integer code is in this coordinate row?” The four components are orthogonal.

XYZAtom::atom_type carries force-field type codes and is always positive. species_code is a separate higher-level column; negative values there do not conflict with atom_type.